

Лабораторная работа №5

Вероятностные алгоритмы проверки чисел на простоту

Кюнкриков Даниил Саналович (НПИМд-01-24, 1132249574)

Российский университет дружбы народов (РУДН), Москва, Россия

Цель работы: реализовать вероятностные алгоритмы проверки чисел на простоту на Python.

Задачи: - тест Ферма; - вычисление символа Якоби; - тест Соловья–Штрассена; - тест Миллера–Рабина; - демонстрация работы программы.

Параметры: - n — проверяемое число; - t — число раундов; - a — случайное основание (свидетель).

Алгоритм: 1) выбрать случайное a ; 2) проверить условие; 3) если условие нарушено — n составное.

Для простого n (и $\gcd(a, n) = 1$):

$$a^{n-1} \equiv 1 \pmod{n}.$$

Если $a^{n-1} \not\equiv 1 \pmod{n}$, то n точно составное.

Реализация: тест Ферма

```
# тест Ферма
def fermat_test_single(n: int) -> str:
    a = random.randint(2, n - 2)
    r = pow(a, n - 1, n)
    return "possible prime" if r == 1 else "composite"

def run_feramn_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >= 5"
    if n % 2 == 0:
        return "composite even"
    for _ in range(t):
        if fermat_test_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Рисунок 1: Программная реализация теста Ферма

Символ Якоби $(a/n) \in \{-1, 0, 1\}$ вычисляется по правилам: - вынесение степеней 2 из a ; - квадратичная взаимность (смена знака при $a \equiv n \equiv 3 \pmod{4}$).

```
# вычисление символа Якоби
def jacobi(a:int, n: int) -> int:
    if n <= 0 or n % 2 == 0:
        raise ValueError("n должно быть нечетным и положительным")
    a %= n
    result = 1
    while a != 0:
        while a % 2 == 0:
            a //= 2
            r = n % 8
            if r == 3 or r == 5:
                result = -result
        a, n = n, a

        if a % 4 == 3 and n % 4 == 3:
            result = -result
        a %= n

    return result if n == 1 else 0
```

Рисунок 2: Программная реализация алгоритма расчета символа Якоби

Критерий Эйлера для простого n :

$$a^{(n-1)/2} \equiv (a/n) \pmod{n}.$$

Если равенство не выполняется — n составное. Оценка ошибки: $\leq 2^{-t}$.

```
# тест Соловья-Штрассена
def solov_strass_test_single(n: int) -> str:
    a = random.randint(2, n-2)
    r = pow(a, (n-1) // 2, n)
    if r != 1 and r != n-1:
        return "composite"

    s = jacobi(a,n)
    return "possible prime" if r == (s+n) % n else "composite"

def run_solov_strass_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >=5"
    if n%2 == 0:
        return "composite even"
    for _ in range(t):
        if solov_strass_test_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Рисунок 3: Программная реализация теста Соловья–Штрассена

$$n - 1 = 2^s \cdot d, \text{ } d \text{ нечётное.}$$

Проверяем цепочку квадратов: - $y_0 = a^d \bmod n$, - $y_{k+1} = y_k^2 \bmod n$.

Оценка ошибки: $\leq 4^{-t}$.

```
# тест Миллера-Рабина

def miller_rabin_test_single(n: int) -> str:
    s = 0
    r = n-1
    while r%2 == 0:
        s += 1
        r //= 2
    a = random.randint(2, n-2)
    y = pow(a, r, n)

    if y != 1 and y != n-1:
        j = 1
        while j <= s - 1 and y != n - 1:
            y = pow(y, 2, n)
            if y == 1:
                return "composite"
            j += 1
        if y != n - 1:
            return "composite"
    return "possible prime"

def run_miller_rabin_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >=5"
    if n%2 == 0:
        return "composite even"
    for _ in range(t):
        if miller_rabin_test_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Рисунок 4: Программная реализация теста Миллера-Рабина


```
введи n: 131
введи t: 20
тест Ферма:           possible prime
тест Соловья-Штрассена: possible prime
тест Миллера-Рабина:  possible prime
```

Рисунок 5: Результат работы вкряточностных тестов простоты

- Реализованы вероятностные тесты простоты: Ферма, Соловэй–Штрассен, Миллер–Рабин.
- Реализовано вычисление символа Якоби.
- Проведена демонстрация работы при различных n и t .